courseCS.WordPress.com

Lesson-1-Features-and-Benefits-of-JavaScript

Download Lesson

Next Lesson

JavaScript is a high-level, versatile, and widely-used programming language primarily known for its role in web development. It's a core technology for building dynamic and interactive websites and web applications. JavaScript, often abbreviated as JS, is a client-side scripting language, meaning it runs in a user's web browser.

Here are some key features and uses of JavaScript:

Web Development: JavaScript is an integral part of web development, allowing developers to enhance the functionality and interactivity of websites. It can be used for tasks such as validating user input, animating elements, and making asynchronous requests to servers.

Client-Side Scripting: JavaScript is executed on the client's device (browser), making it ideal for creating interactive web interfaces. It can respond to user actions, such as button clicks and form submissions, without requiring a full page reload.

Cross-Browser Compatibility: JavaScript is supported by all major web browsers, making it a cross-browser language. This ensures that web applications work consistently across different browsers and platforms.

Versatility: JavaScript is a versatile language that can be used for various purposes beyond web development. It can be used for server-side development (Node.js), mobile app development (React Native), and even in hardware and IoT applications.

Dynamic Typing: JavaScript is a dynamically typed language, meaning you don't need to declare data types explicitly. Variables can change data types at runtime, offering flexibility but requiring careful coding practices.

Objects and Functions: JavaScript is an object-oriented language, and almost everything in JavaScript is an object, including functions. Functions are first-class citizens, meaning they can be assigned to variables, passed as arguments, and returned as values.

Asynchronous Programming: JavaScript supports asynchronous programming using Promises and Async/Await. This allows developers to handle time-consuming operations, such as fetching data from servers, without blocking the main thread.

Community and Ecosystem: JavaScript has a vast ecosystem of libraries and frameworks that simplify common tasks. Popular front-end libraries and frameworks include React, Angular, and Vue.js. On the back end, Node.js is widely used for server-side development.

Security: JavaScript execution is limited by the same-origin policy, which restricts scripts from making requests to different domains to prevent security vulnerabilities. Web security practices, like input validation and output encoding, are essential when using JavaScript.

Open Standard: JavaScript is an open standard and part of the web technologies defined by the World Wide Web Consortium (W3C). It's governed by a set of standards (ECMAScript), ensuring consistency and interoperability.

JavaScript is a powerful and essential tool for creating dynamic and interactive web applications. Its ubiquity and flexibility have made it a fundamental skill for web developers and a cornerstone of modern web development.

Next Lesson

.

.

.

.

.

.

.

.

.

.

.

.

.

.

.

courseCS.WordPress.com

Lesson-2-Variables-and-their-Declaration

Download Lesson

Previous Lesson Next Lesson

Variables are containers for storing values. JavaScript Variables can be declared in 4 ways

1. Automatically
2. Using `var`
3. Using `let`
4. Using `const`

2.1. Automatically Variable Declaration

Example Code 1: Variables are declared automatically without any data type

```
//variable declaration and initialization
x = 5; //integral type variable
z = 5.5; //float type variable
w = "Web Development" //string type variable
t = 'Web Development' //string initialize with single quotation marks
a = "123" //it is string instead of number

//displaying values of above variables
console.log("Value of x")
console.log(x)
console.log("Value of z")
console.log(z)
console.log("Value of w")
console.log(w)
console.log("Value of t")
console.log(t)
console.log("Value of a")
console.log(a)
```

Note: Good programming practice is to always declare variables before their use.

2.2. Using "var"

The "`var`" keyword was used in all JavaScript code from 1995 to 2015. It should only be used in code written for older browsers. In other words, only use "`var`" if you MUST support old browsers.

Example Code 2:

- Line 9 will not generate error

```
1  //variable declaration and initialization
2  var x = 5; //integral type variable
3  var z = 5.5; //float type variable
4  var w = "Web Development" //string type variable
5  var t = 'Web Development' //string initialize with single quotation marks
6  var a = "123" //it is string instead of number
7
8  //Updating value of w
9  w = 12
10
11 //displaying values of above variables
12 console.log("Value of x")
13 console.log(x)
14 console.log("Value of z")
15 console.log(z)
16 console.log("Value of w")
17 console.log(w)
18 console.log("Value of t")
19 console.log(t)
20 console.log("Value of a")
21 console.log(a)
```

2.3. Using "const"

The "`const`" keywords is added to JavaScript in 2015. Always use "`const`" if the value should not be changed. Always use "`const`" if the type should not be changed (Arrays and Objects)

Example Code 3:

- Line 9 will generate error because variable "t" is declared with "const". The value of such type of variables cannot be updated.

```
1  //variable declaration and initialization
2  const x = 5; //integral type variable
3  const z = 5.5; //float type variable
4  const w = "Web Development" //string type variable
5  const t = 'Web Development' //string initialize with single quotation marks
6  const a = "123" //it is string instead of number
7
8  //Error Line: cannot be updated
9  t = 12
10
11 //displaying values of above variables
12 console.log("Value of x")
13 console.log(x)
14 console.log("Value of z")
15 console.log(z)
16 console.log("Value of w")
17 console.log(w)
18 console.log("Value of t")
19 console.log(t)
20 console.log("Value of a")
21 console.log(a)
```

2.4. Using “let”

The “`let`" keywords is added to JavaScript in 2015 along with “const”. Always use “`let`" if the value can be changed.

Example Code 4:

- “var” and “let” are same. But “let” is supported by old browser. In latest (after 2015) browser, both “let” and “const” are used
- Line 9 will generate error because variable “t” is declared with “const”. The value of such type of variables cannot be updated.

```
1  //variable declaration and initialization
2  let x = 5; //integral type variable
3  let z = 5.5; //float type variable
4  let w = "Web Development" //string type variable
5  let t = 'Web Development' //string initialize with single quotation marks
6  let a = "123" //it is string instead of number
7
8  //Updating value of w
9  w = 12
10
11 //displaying values of above variables
12 console.log("Value of x")
13 console.log(x)
14 console.log("Value of z")
15 console.log(z)
16 console.log("Value of w")
17 console.log(w)
18 console.log("Value of t")
19 console.log(t)
20 console.log("Value of a")
21 console.log(a)
```

Note: It’s a good programming practice to declare all variables at the beginning of a script.

Note: It's a good programming practice to comment your code's sections as much as possible

JavaScript Identifier

All JavaScript **variables** must be **identified** with **unique names**. These unique names are called **identifiers**.

Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume). The general rules for constructing names for variables (unique identifiers) are:

- Names can contain letters, digits, underscores, and dollar signs.
- Names must begin with a letter.
- Names can also begin with $ and _ (but we will not use it in this tutorial).
- Names are case sensitive (y and Y are different variables).
- Reserved words (like JavaScript keywords) cannot be used as names.

Note: JavaScript identifiers are case-sensitive.

Previous Lesson **Next Lesson**

.

.

.

.

.

.

.

.

.

.

.

.

.